JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Interface AsynchronousByteChannel

All Superinterfaces:

`AsynchronousChannel`, `AutoCloseable`, `Channel`, `Closeable`

All Known Implementing Classes:

`AsynchronousSocketChannel`

```
public interface AsynchronousByteChannel
extends AsynchronousChannel
```

An asynchronous channel that can read and write bytes.

Some channels may not allow more than one read or write to be outstanding at any given time. If a thread invokes a read method before a previous read operation has completed then a `ReadPendingException` will be thrown. Similarly, if a write method is invoked before a previous write has completed then `WritePendingException` is thrown. Whether or not other kinds of I/O operations may proceed concurrently with a read operation depends upon the type of the channel.

Note that `ByteBuffers` are not safe for use by multiple concurrent threads. When a read or write operation is initiated then care must be taken to ensure that the buffer is not accessed until the operation completes.

Since:

1.7

See Also:

`Channels.newInputStream(AsynchronousByteChannel)`, `Channels.newOutputStream(AsynchronousByteChannel)`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
`Future<Integer>`	`read(ByteBuffer dst)`	Reads a sequence of bytes from this channel into the given buffer.
`<A> void`	`read(ByteBuffer dst, A attachment, CompletionHandler<Integer,? super A> handler)`	Reads a sequence of bytes from this channel into the given buffer.
`Future<Integer>`	`write(ByteBuffer src)`	Writes a sequence of bytes to this channel from the given buffer.
`<A> void`	`write(ByteBuffer src, A attachment, CompletionHandler<Integer,? super A> handler)`	Writes a sequence of bytes to this channel from the given buffer.

Methods inherited from interface java.nio.channels.AsynchronousChannel

close

Methods inherited from interface java.nio.channels.Channel

isOpen

Method Detail

read

```
<A> void read(ByteBuffer dst,
              A attachment,
              CompletionHandler<Integer,? super A> handler)
```

Reads a sequence of bytes from this channel into the given buffer.

This method initiates an asynchronous read operation to read a sequence of bytes from this channel into the given buffer. The `handler` parameter is a completion handler that is invoked when the read operation completes (or

fails). The result passed to the completion handler is the number of bytes read or `-1` if no bytes could be read because the channel has reached end-of-stream.

The read operation may read up to *r* bytes from the channel, where *r* is the number of bytes remaining in the buffer, that is, `dst.remaining()` at the time that the read is attempted. Where *r* is 0, the read operation completes immediately with a result of `0` without initiating an I/O operation.

Suppose that a byte sequence of length *n* is read, where `0` < *n* <= *r*. This byte sequence will be transferred into the buffer so that the first byte in the sequence is at index *p* and the last byte is at index *p* + *n* - 1, where *p* is the buffer's position at the moment the read is performed. Upon completion the buffer's position will be equal to *p* + *n*; its limit will not have changed.

Buffers are not safe for use by multiple concurrent threads so care should be taken to not access the buffer until the operation has completed.

This method may be invoked at any time. Some channel types may not allow more than one read to be outstanding at any given time. If a thread initiates a read operation before a previous read operation has completed then a `ReadPendingException` will be thrown.

Type Parameters:

`A - The type of the attachment`

Parameters:

`dst - The buffer into which bytes are to be transferred`

`attachment - The object to attach to the I/O operation; can be null`

`handler - The completion handler`

Throws:

`IllegalArgumentException - If the buffer is read-only`

`ReadPendingException - If the channel does not allow more than one read to be outstanding and a previous read has not completed`

`ShutdownChannelGroupException - If the channel is associated with a group that has terminated`

read

`Future<Integer> read(ByteBuffer dst)`

Reads a sequence of bytes from this channel into the given buffer.

This method initiates an asynchronous read operation to read a sequence of bytes from this channel into the given buffer. The method behaves in exactly the same manner as the `read(ByteBuffer,Object,CompletionHandler)`

method except that instead of specifying a completion handler, this method returns a `Future` representing the pending result. The `Future`'s `get` method returns the number of bytes read or `-1` if no bytes could be read because the channel has reached end-of-stream.

Parameters:

`dst - The buffer into which bytes are to be transferred`

Returns:

`A Future representing the result of the operation`

Throws:

`IllegalArgumentException - If the buffer is read-only`

`ReadPendingException - If the channel does not allow more than one read to be outstanding and a previous read has not completed`

write

```
<A> void write(ByteBuffer src,
               A attachment,
               CompletionHandler<Integer,? super A> handler)
```

Writes a sequence of bytes to this channel from the given buffer.

This method initiates an asynchronous write operation to write a sequence of bytes to this channel from the given buffer. The `handler` parameter is a completion handler that is invoked when the write operation completes (or fails). The result passed to the completion handler is the number of bytes written.

The write operation may write up to *r* bytes to the channel, where *r* is the number of bytes remaining in the buffer, that is, `src.remaining()` at the time that the write is attempted. Where *r* is 0, the write operation completes immediately with a result of `0` without initiating an I/O operation.

Suppose that a byte sequence of length *n* is written, where `0` < *n* <= *r*. This byte sequence will be transferred from the buffer starting at index *p*, where *p* is the buffer's position at the moment the write is performed; the index of the last byte written will be *p* + *n* `-` `1`. Upon completion the buffer's position will be equal to *p* + *n*; its limit will not have changed.

Buffers are not safe for use by multiple concurrent threads so care should be taken to not access the buffer until the operation has completed.

This method may be invoked at any time. Some channel types may not allow more than one write to be outstanding at any given time. If a thread initiates a write operation before a previous write operation has completed then a `WritePendingException` will be thrown.

Type Parameters:

A - The type of the attachment

Parameters:

src - The buffer from which bytes are to be retrieved

attachment - The object to attach to the I/O operation; can be null

handler - The completion handler object

Throws:

WritePendingException - If the channel does not allow more than one write to be outstanding and a previous write has not completed

ShutdownChannelGroupException - If the channel is associated with a group that has terminated

write

Future<Integer> write(ByteBuffer src)

Writes a sequence of bytes to this channel from the given buffer.

This method initiates an asynchronous write operation to write a sequence of bytes to this channel from the given buffer. The method behaves in exactly the same manner as the write(ByteBuffer,Object,CompletionHandler) method except that instead of specifying a completion handler, this method returns a Future representing the pending result. The Future's get method returns the number of bytes written.

Parameters:

src - The buffer from which bytes are to be retrieved

Returns:

A Future representing the result of the operation

Throws:

`WritePendingException` - If the channel does not allow more than one write to be outstanding and a previous write has not completed

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.